

Sky Analytics Engine

Analisi batch e Metriche del Traffico Aereo

Flavio Simonelli Francesco Masci

Corso Architettura e Sistemi per Big Data
Corso di Laurea Magistrale in Ingegneria Informatica
Università di Roma "Tor Vergata"

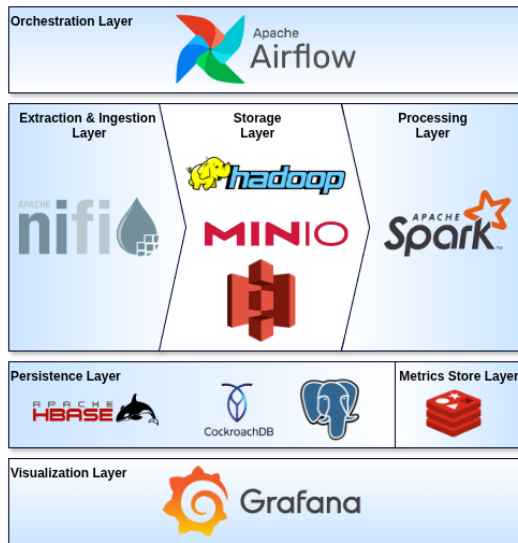
12 Giugno 2026



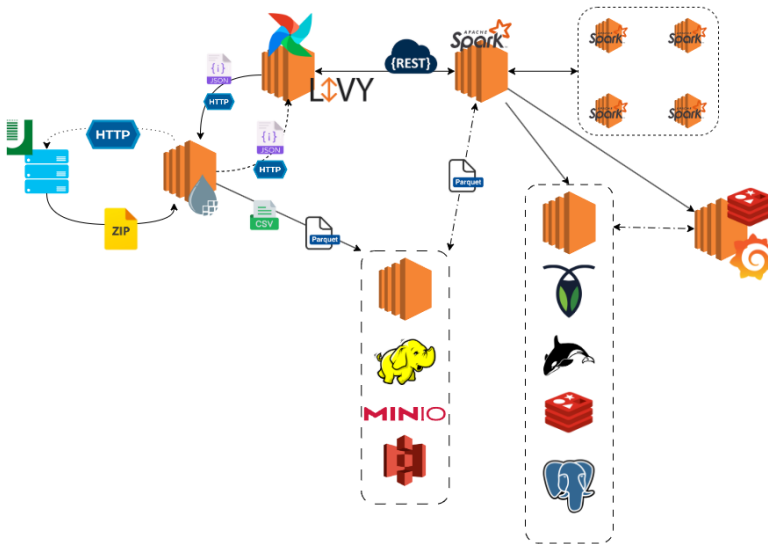
TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

- 1 Big Data Stack
- 2 Flow Diagram
- 3 Sequence Diagram
- 4 Extraction & Ingestion Layer
- 5 Computation Layer
- 6 Airflow Orchestration
- 7 Deployment
- 8 Valutazione Sperimentale: Metriche

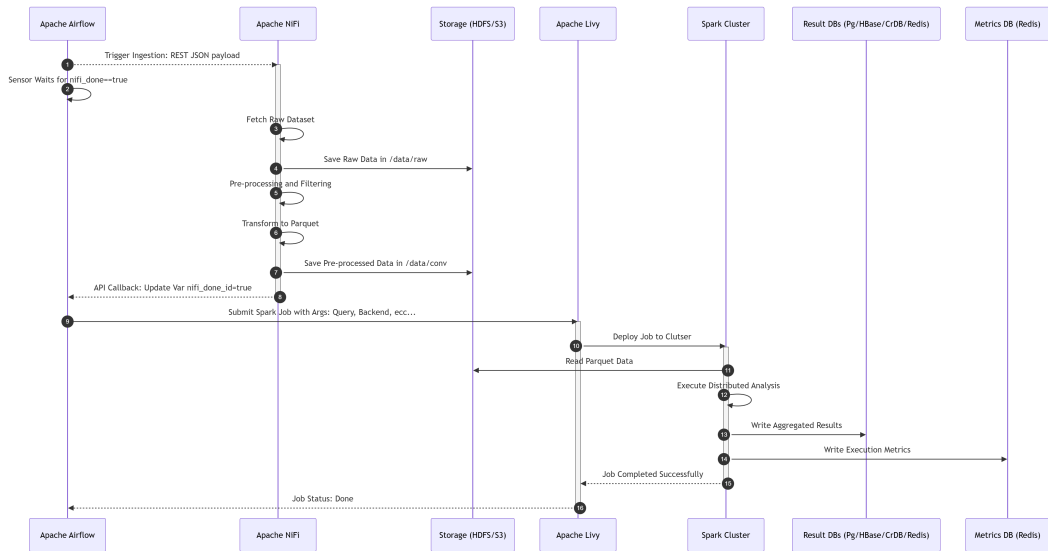
Big Data Stack



Flow Diagram



Sequence Diagram



NiFi Ingestion Pipeline: Approccio Data-Driven

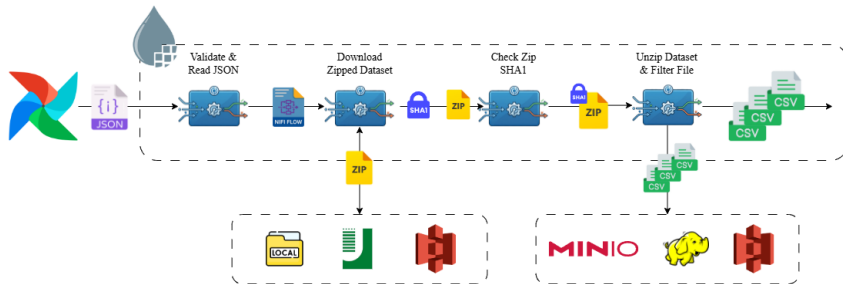
- **Design Data-Driven:** Pipeline stateless configurata dinamicamente tramite payload JSON iniettato all'avvio.
- **Ingestion & Validazione:** `HandleHttpRequest` espone l'endpoint REST per l'avvio da Airflow, mentre `ValidateJson` esegue un controllo strutturale con schema rigoroso.
- **Propagazione del Dato:** `EvaluateJsonPath` mappa le chiavi JSON in Attributi del FlowFile, guidando in modo stateless il routing e le decisioni future tramite metadati nativi.

Payload JSON

```
{
  "job_id": "FLIGHT_INGEST_run_123",
  "storage_raw": {
    "type": "HDFS",
    "path": "/data/raw"
  },
  "storage_preprocessed": {
    "type": "S3",
    "bucket": "spark-flight-analysis",
    "path": "data/conv"
  },
  "processing": {
    "optimization_strategy": "PARTITION..."
  },
  "callback": {
    "url": "http://airflow.../variables/v",
    "method": "PATCH",
    "headers": {
      "Authorization": "Bearer eyJhbG..."
    }
  }
}
```

Ingestion, Sicurezza e Gateway

- **Estrazione:** Ricezione e spaccettamento degli archivi (UnpackContent).
- **Validazione SHA-1:**
 - Controllo di integrità crittografica sui file estratti.
 - Confronto hash atteso vs calcolato (CryptographicHashContent).
 - Routing degli errori per isolare dati corrotti.
- **Gateway & Biforcazione:**
 - **Ramo Raw:** Salvataggio as-is per garanzia del dato originale.
 - **Ramo Preprocessing:** Inoltro dei record validati alla trasformazione.



Preprocessing e Ottimizzazione in Parquet

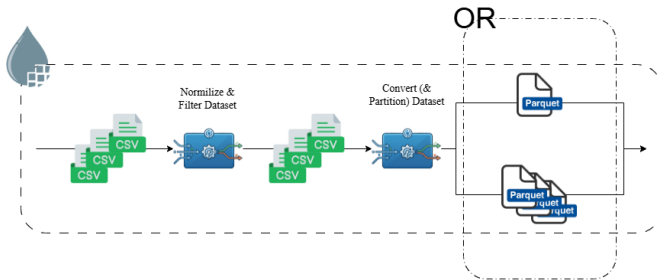
- **Operatori NiFi per la Trasformazione:**

- QueryRecord/UpdateRecord: Filtraggio, casting e normalizzazione.
- LookupRecord: Arricchimento dei dati via file dizionario.
- MergeRecord: Prevenzione dello "Small Files Problem".

- **Target Parquet:** Conversione con schema enforcement (Avro).

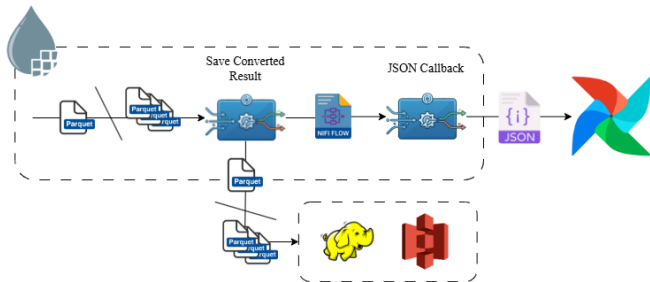
- **Strategie di Ottimizzazione:**

- **Partition Pruning:** Generazione dinamica dei path (UpdateAttribute).
- **Predicate Pushdown:** Ottimizzazione dei metadati per Spark.



Storage Finale e Handover ad Airflow

- **Storage Distribuito:** Scrittura dei file Parquet ottimizzati in HDFS o S3.
- **Notifica di Completamento:**
 - Chiamata REST via `InvokeHTTP` (PATCH/POST) alle API di Airflow.
 - Handover del payload aggiornato con lo stato dell'elaborazione.
- **Orchestrazione Analitica:** Sblocco automatico dei successivi job Spark in base alla strategia scelta.



- **Ecosistema Spark:** Implementazione in Java utilizzando Spark Core (RDD), Spark SQL e MLlib.
- **Design Pattern Template Method:**
 - Astrazione tramite la classe `BaseQuery.java`.
 - Garanzia di coerenza tra i diversi paradigmi di esecuzione.
- **Benchmarking Comparativo:** Ogni analisi implementa obbligatoriamente tre motori:
 - **RDD API:** Controllo granulare su partizioni e serializzazione.
 - **DataFrame API:** Astrazione strutturata (DSL) e ottimizzazione *Catalyst*.
 - **Spark SQL:** Interfaccia dichiarativa per logiche di business complesse.

Query 1: Analisi Mensile delle Performance

- **Obiettivo:** Ritardi medi, estremi e tasso di cancellazione mensile per vettore.
- **RDD (aggregateByKey):**
 - `pairs.aggregateByKey(zeroValue, seqOp, combOp)`
 - Aggregazione locale su partizione per minimizzare il traffico di rete.
- **DataFrame (DSL):**
 - `flights.groupBy("MONTH", "CARRIER").agg(avg("DELAY"), ...)`
- **Spark SQL:**
 - `SELECT MONTH, AVG(DELAY) FROM flights GROUP BY MONTH, ...`

Query 2: Ranking dei Ritardi in Arrivo

- **Obiettivo:** Identificazione delle Top-K tratte più critiche.
- **RDD (Manuale):**
 - `mapToPair().reduceByKey().takeOrdered(10, comparator)`
- **DataFrame (Window Functions):**
 - `agg(avg("ARR_DELAY")).orderBy(col("mean").desc()).limit(10)`
- **Spark SQL:**
 - `SELECT carrier, AVG(ARR_DELAY) as mean FROM flights ... ORDER BY mean DESC LIMIT 10`

Query 3: Distribuzione e Percentili

- **Obiettivo:** Statistiche orarie (Q1, Mediana, Q3).
- **RDD (La sfida):** `groupByKey()` su milioni di record causerebbe OOM.
- **DataFrame / SQL (Approssimazione):**
 - `agg(expr("percentile_approx(DEP_DELAY, 0.5)"))`
 - Uso dell'algoritmo nativo Greenwald-Khanna per stime efficienti.
- **Soluzione RDD Custom:**
 - `PercentileSketch sketch = PercentileSketch.from(config)`
 - Integrazione di T-Digest e KLL per calcolo distribuito e mergeable.

Gestione Percentili in RDD: Strategie di Sketching

- **Design Pattern Strategy:** Implementazione di algoritmi probabilistici in RDD.
- **T-Digest Strategy:**
 - Struttura dati basata su centroidi pesati.
 - Alta precisione nelle "code" della distribuzione (ritardi estremi).
- **KLL Sketch Strategy:**
 - Algoritmo stream-based con footprint di memoria fisso e predicibile.
 - Merge delle strutture dati tra worker senza trasferimento di interi dataset.

Machine Learning: Airline Clustering

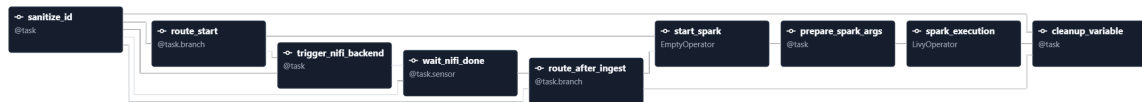
- **Obiettivo:** Raggruppamento di vettori aerei con profili operativi simili.
- **Modello K-Means** (*Spark MLlib*):
 - Inizializzazione tramite `k-means||` (variante parallela di `k-means++`).
 - Convergenza distribuita sui centroidi dei cluster.
- **Data Pipeline:**
 - **VectorAssembler:** Consolidamento delle features analitiche.
 - **StandardScaler:** Normalizzazione fondamentale per la distanza Euclidea.

Telemetria Netta: Custom Spark Listener

- **Obiettivo:** Misurazione rigorosa dei tempi di esecuzione per il benchmarking.
- **JobTimerListener:**
 - Estensione di SparkListener per intercettare gli eventi del DAG Scheduler.
 - Monitoraggio di onJobStart e onJobEnd.
- **Precisione:** Eliminazione degli overhead di sistema, I/O Driver e task "lazy", garantendo metriche focalizzate puramente sul tempo di calcolo distribuito.

Orchestrazione End-to-End: flight_analysis

- **Obiettivo:** Orchestrare l'intera pipeline, dall'ingestion in NiFi all'elaborazione Spark.
- **Funzionalità:** Utilizzo della **TaskFlow API** (Airflow 3.0) per gestire il branching condizionale (*Ingest Only, Execution Only, Both*).
- **Sincronizzazione:** Uso di `Task.Sensor` per attendere il completamento asincrono di NiFi.



- **Obiettivo:** Raccolta automatizzata delle metriche prestazionali in ambiente EC2.
- **Logica:** Loop sequenziale per eseguire tutte le 12 combinazioni (4 Query $\times$ 3 Backend).
- **Integrazione:** Sottomissione dei job tramite LivyOperator verso il Master Node.

Integrazione Livy (EC2)

```
spark_task = LivyOperator(  
    task_id=task_id,  
    livy_conn_id="livy_default",  
    file=JAR_PATH,  
    class_name=JAR_CLASS,  
    args=[  
        "--query", query,  
        "--backend", backend,  
        "--config", "{{ params.config_file }}",  
        "--input-type", "{{ params.input_type }}",  
        "--output-type", "{{ params.output_type }}",  
        "--metrics-type", "{{ params.metrics_type }}"  
    ],  
    name=f"benchmark-{query}-{backend}",  
    polling_interval=5  
)
```

Benchmarking su AWS Managed: emr_benchmark

- **Obiettivo:** Benchmarking su **AWS EMR** per testare la scalabilità managed.
- **Logica:** Esecuzione delle 12 combinazioni in modalità sequenziale.
- **Integrazione Cloud:** Uso di `EmrAddStepsOperator` e `EmrStepSensor`.

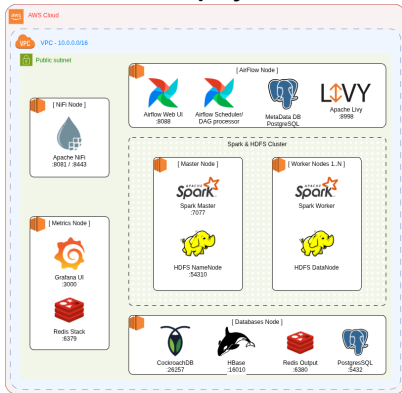
Integrazione EMR Step (AWS)

```
spark_step = {
  "Name": step_name,
  "ActionOnFailure": "CONTINUE",
  "HadoopJarStep": {
    "Jar": "command-runner.jar",
    "Args": [
      "spark-submit", "--deploy-mode", "client",
      "--class", JAR_CLASS, JAR_PATH,
      "--query", query, ...
    ],
  },
}

add_step = EmrAddStepsOperator(
  task_id=tid,
  job_flow_id="{{ params.job_flow_id }}",
  aws_conn_id="aws_default",
  steps=[spark_step],
)

watch_step = EmrStepSensor(
  task_id=task_id,
  job_flow_id="{{ params.job_flow_id }}",
  step_id=f"{{{ ti.xcom_pull(task_ids='{tid}') [0] }}}",
  aws_conn_id="aws_default",
  poke_interval=30,
)
```

EC2 Deployment



EMR Deployment

[Immagine in arrivo]

- **Rigore Statistico:** Ogni combinazione (Query $\times$ Backend) è stata eseguita **5 volte** per garantire la stabilità delle misurazioni.
- **Telemetria:** Utilizzo del JobTimerListener (SparkListener) per isolare il *tempo netto di calcolo* dall'overhead di sistema.
- **Hardware e Infrastruttura:**
 - **EC2 (Self-Hosted):** Master + Worker su istanze t3.small. Storage basato su HDFS.
 - **EMR (Cloud-Native):** 1 Master + 3 Core su istanze m6g.xlarge (ARM64). Architettura *Shared-Nothing* su Amazon S3 via s3a://.

Confronto API: RDD vs DataFrame vs SQL

- **DataFrame/SQL:** Massima stabilità e minori tempi di esecuzione nelle query di aggregazione (es. Q1) grazie all'ottimizzatore *Catalyst* e alla gestione binaria off-heap di *Tungsten*.
- **RDD:** Più performante in task atomici e logiche distribuite custom (Q2 e Q3), nonostante il maggior overhead di serializzazione JVM.



Ottimizzazione I/O: Pruning vs Pushdown

- **Partition Pruning:** Strategia ottimale. Esclude i dati a livello di file system, garantendo un I/O bilanciato e l'assenza di *Workload Skew* sui nodi.
- **Predicate Pushdown:** Richiede la scansione dei metadati Parquet. Genera un volume di lettura superiore e tende a sovraccaricare singoli nodi (colli di bottiglia in fase di scansione).



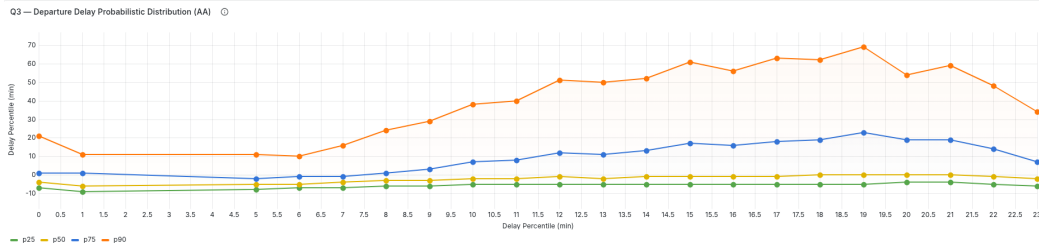
Overhead Infrastrutturale: EC2 vs EMR

- **Standalone (EC2):** Overhead di *Wall Clock Time* significativo dovuto ai tempi di negoziazione delle risorse, bootstrap del cluster e latenze di rete HDFS.
- **Managed (EMR):** L'integrazione nativa con S3 e la gestione elastica delle istanze contraggono sensibilmente i "tempi morti", massimizzando l'efficienza temporale del job.



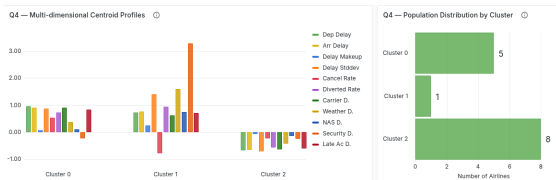
Query 3: Scelta dell'Euristica e Approssimazione

- **Problema:** Calcolo dei percentili esatti su RDD (groupByKey) causa *Out Of Memory* e colli di bottiglia distribuiti.
- **Soluzione:** Implementazione di algoritmi di **Quantile Sketching** (T-Digest e KLL).
- **Analisi Comparativa:**
 - **KLL Sketch:** Elevata precisione teorica, ma footprint di memoria superiore.
 - **T-Digest:** Ottimizzato per le "code" della distribuzione e più performante.
- **Validazione:** Risultati pressoché identici sul dataset reale; la convergenza dei dati giustifica l'uso di algoritmi probabilistici.
- **Scelta Finale:** **T-Digest** per il miglior rapporto performance/accuratezza nel dominio aeronautico.

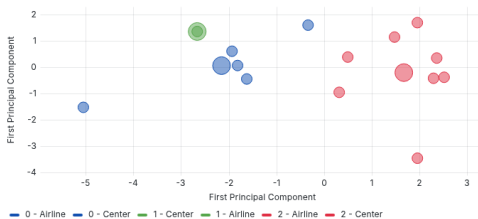


Risultati Analitici: Airline Clustering (Q4)

- **Model Selection:** Identificazione di $K = 3$ come numero ottimale di cluster tramite *Silhouette Score*.
- **Profilazione dei Cluster:**
 - **Cluster 0 (Inefficienza):** AA, OH, F9. Elevata propagazione dei ritardi e instabilità operativa.
 - **Cluster 1 (Sensibilità Esterna):** G4. Forti ritardi meteorologici ma bassissimo tasso di cancellazione.
 - **Cluster 2 (Virtuosi):** DL, UA, WN. Alta capacità di recupero in volo (*Delay Makeup*) e stabilità.
- **Visualizzazione:** Proiezione PCA bidimensionale (perdita di varianza fisiologica rispetto al modello ad alta dimensionalità).



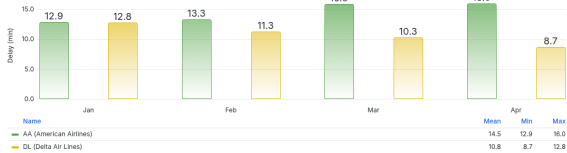
Q4 — Latent Performance Clusters (PCA Projection)



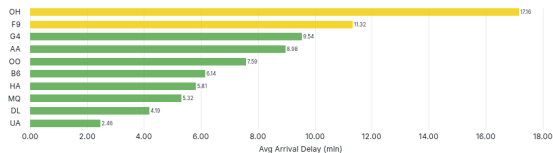
Risultati Analitici: Query 1 e Query 2

- **Query 1 (Trend):** Divergenza tra **Delta** (miglioramento costante, ritardi < 10 min) e **American Airlines** (degrado progressivo e maggiori cancellazioni).
- **Query 2 (Ranking):**
 - **PSA Airlines (OH):** Peggior vettore per ritardo all'arrivo (17.16 min), causato da forte propagazione (*late aircraft delay*).
 - **United (UA):** Più efficiente tra i grandi vettori, ma penalizzato da fattori esterni (*NAS delay*).
- **Insight:** SAE decodifica con precisione le inefficienze strutturali vs eventi esterni.

Q1 — Average Departure Delay Trend (AA vs DL)



Q2 — Top 10 Delayed Airlines (Arrival)



Conclusioni

- **Modularità:** L'architettura disaccoppiata (NiFi-Airflow-Spark) garantisce scalabilità ed estensibilità.
- **Efficienza:** Il formato Parquet e il Partition Pruning sono fondamentali per abbattere l'I/O overhead.
- **Insight:** La piattaforma trasforma dataset massivi in profili operativi chiari per il business.
- **Sviluppi Futuri:** Transizione verso un'architettura Lambda per il processamento in streaming real-time.



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA